C

```c
#include <stdio.h>
#include<string.h>
#include <stdlib.h>

// union Student{
//     int roll_no;
//     char name[50];
//     float marks;
// };

int main(){


    int *ptr;

    // ptr = (int*) malloc(sizeof(int)*5 ); // 20
    ptr = (int*) calloc(5, sizeof(int)); // 20

    for(int i=0; i<5; i++){
        // ptr[i] = i+1;
        printf("%d \n", ptr[i]);
    }

    // ptr = (int *) realloc(ptr, sizeof(int) * 10);

    // for(int i=5; i<10; i++){

    //     printf("%d \n", ptr[i]);
    // }

    // free(ptr);


    // FILE *file;

    // file = fopen("example.txt", "a");

    // if(file == NULL){
    //     printf("FIle not created \n");
    // }
    // fprintf(file, "C  Dennis ritchie \n");
    // fprintf(file, "1972 \n");

    // fclose(file);

    // FILE *file1 = fopen("example.txt", "r");
```

```
    // if(file1 == NULL){
    //     printf("File does not exist \n");
    // }
    // char data[255];

    // while( fgets(data , 255, file1) != NULL){

    //     printf("%s\n", data);
    // }


    // printf("Union \n");

    // union Student s1;



    // s1.marks = 98;
    // strcpy(s1.name , "Harish");
    // s1.roll_no = 21;

    // printf("%d \n", s1.roll_no);
    // printf("%s \n", s1.name);

    // printf("%f \n", s1.marks);



    return 0;
}
```

📘 Theory

- **Dynamic Memory Allocation**
  - malloc(size) → allocates memory but does not initialize it.
  - calloc(n, size) → allocates memory and initializes all bytes to zero.
  - realloc(ptr, new_size) → resizes previously allocated memory.
  - free(ptr) → deallocates memory.
- **File Handling**
  - fopen("file.txt", "w") → open file for writing.
  - fopen("file.txt", "r") → open file for reading.
  - fprintf(file, "text") → write to file.

    - fgets(buffer, size, file) → read from file.
    - Always check if file == NULL to avoid runtime errors.
- **Union**
    - Similar to struct but shares memory among all members.
    - Only one member can hold a value at a time.
    - Useful for memory-efficient storage.

📝 **Coding Round Questions**

- Malloc vs Calloc → Write a program to allocate memory using malloc and calloc, initialize values, and print them.
- Realloc usage → Demonstrate how to expand an array dynamically using realloc.
- File write/read → Write a program that stores student details in a file and then reads them back.
- Union vs Struct → Create a union Student and a struct Student, assign values, and show how memory usage differs.
- Free memory → Write a program that allocates memory for 10 integers, frees it, and shows why accessing freed memory is unsafe.

⚡ **Machine Coding Round Challenge**

Problem: Student Records with Dynamic Memory and File Handling

Requirements:

1. Use malloc to allocate memory for n students.
2. Each student has roll_no, name, and marks.
3. Store student details in a file (students.txt).
4. Read back the file and display student details.
5. Use realloc to expand the array if more students are added.
6. Demonstrate union by storing either roll_no or marks in the same memory block.